

Algorithmes et Pensée Computationnelle

Programmation orientée objet : Héritage et Polymorphisme

Le but de cette séance est d'approfondir les notions de programmation orientée objet vues précédemment. Nous commencerons par un rappel des notions de surcharge d'opérateurs avant d'introduire des exercices sur l'héritage et le polymorphisme. Au terme de cette séance, vous devez être en mesure de factoriser votre code afin de le rendre mieux structuré et plus lisible. À chaque exercice, le langage de programmation à utiliser sera spécifié.

Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `Code`.

1 Rappel : Surcharge des opérateurs - Python

Dans cette section, vous manipulerez des fractions sous forme d'objets. Vous ferez des opérations de base sur ce nouveau type d'objets.

Question 1: (🕒 5 minutes) Création de classe

Dans un projet que vous aurez au préalable préparé, créez un fichier nommé `surcharge.py`. À l'intérieur de ce fichier, créez une classe `Fraction` qui aura comme attributs privés un numérateur et un dénominateur.

Question 2: (🕒 5 minutes) Constructeur par défaut

Définir un constructeur à votre classe. Assignez des valeurs par défaut à vos attributs.

Si un seul argument est passé à votre constructeur, la fraction devra être égale à l'entier correspondant.

Empêchez l'utilisateur d'assigner la valeur zéro au dénominateur.

Question 3: (🕒 5 minutes) Type casting

Convertir les attributs en entier.

Question 4: (🕒 5 minutes) Redéfinition de méthodes

Redéfinir la méthode `__str__()` pour produire une représentation textuelle de vos objets `Fraction`.

Question 5: (🕒 5 minutes) Accesseurs et mutateurs

Créer des `getters` et `setters` pour chacun des attributs de votre classe `Fraction`.

Question 6: (🕒 10 minutes) Simplification de fractions

Définir une méthode `simplification` qui réduit la `Fraction`. Cette méthode ne renverra rien, elle modifiera simplement l'instance. Pour la suite des exercices, assurez-vous de toujours manipuler des fractions simplifiées. Pour ce faire, vous pouvez faire appel à votre méthode `simplification` après chaque opération sur un objet de type `Fraction`.

Question 7: (🕒 15 minutes) Redéfinition de méthodes - `__eq__`

Redéfinir la méthode d'instance `__eq__` qui prend en entrée un objet `Fraction` que vous nommerez `other` (en plus de `self`) et qui renvoie `True` si `self` et l'objet passé en argument ont la même valeur.

Question 8: (🕒 15 minutes) Addition et multiplication

Redéfinir les méthodes `__add__` et `__mul__` afin d'effectuer des opérations d'addition et de multiplication sur vos objets de type `Fraction`. Attention, ces méthodes devront renvoyer des objets de type `Fraction`. Dans vos méthodes `__add__` et `__mul__`, n'oubliez pas de simplifier ces fractions avant de les retourner. Gérer le cas où l'élément passé en argument n'est ni une `Fraction`, ni un `int`.

2 Polymorphisme - Java

Les exercices de cette section s'inspirent des exercices de la section 1 des exercices avancés de la semaine 6. N'hésitez pas à réutiliser les solutions disponibles sur Moodle.

Dans cette partie, vous devez créer 2 nouvelles classe-filles de la classe-mère `Fighter`. La première classe représentera un `Soigneur`, qui, lorsqu'il "attaquera" un `Fighter`, le soignera au lieu de le blesser. La

deuxième classe représentera un `Fighter` spécialisé dans l'attaque `Attaquant`, qui aura la capacité d'attaquer un certain nombre de fois (ce nombre sera défini lors de l'instanciation).

Avant d'effectuer la série de questions suivantes, téléchargez le fichier `fighter-squelette.java` disponible sur Moodle.

Question 9: (🕒 5 minutes) classe-fille `Soigneur`

Créez une nouvelle classe-fille `Soigneur` qui hérite de la classe `Fighter`. Cette classe-fille aura un nouvel attribut privé nommé `résurrection` qui vaudra 1 lors de l'instanciation de la classe.

Créez le constructeur de cette classe ainsi que les `getter` et `setter` permettant d'interagir avec le nouvel attribut (`résurrection`).

Question 10: (🕒 15 minutes) Méthode `résurrection(Fighter other)` de la classe-fille `Soigneur`

Créez une méthode `résurrection(Fighter other)` qui permettra de faire revenir un `Fighter` à la vie, mais le `Soigneur` ne pourra le faire qu'une seule fois.

Commencez par contrôler que l'instance depuis laquelle la méthode est appelée soit toujours en vie. Si ce n'est pas le cas, indiquez : `nom_instance` est mort et ne peut plus rien faire.

Contrôlez ensuite que l'instance `other` soit vraiment morte. Si ce n'est pas le cas, indiquez le en affichant le texte suivant : `"nom_other est toujours en vie."`

Pour finir, contrôlez que l'attribut `résurrection` de l'instance depuis laquelle la méthode est appelée est égale à 1. Si ce n'est pas le cas, indiquez : `nom_instance` ne peut plus ressusciter personne.

Si tous ces éléments sont réunis, faites revenir le `Fighter other` à la vie en lui remettant 10 points de vie et en l'ajoutant à la liste `instances` de la classe `Fighter`. Pensez aussi à :

- Mettre l'attribut `résurrection` de l'instance appelée à 0 afin de l'empêcher de réutiliser ce pouvoir,
- appeler la méthode `checkHealth()`, et à indiquer : `"nom_other est revenu à la vie!"`

Question 11: (🕒 10 minutes) Méthode `attack` de la classe-fille `Soigneur`

Réécrivez la méthode `attack` de la classe-fille `Soigneur` afin d'ajouter des points de vie à `other` au lieu de lui en retirer.

Le seul argument nécessaire pour cette méthode sera une autre instance de `Fighter` qu'on pourrait nommer `other`.

Commencez par contrôler que le `Soigneur` depuis lequel la méthode est appelée est encore en vie. Si ce n'est pas le cas, affichez : `"nom_instance est mort et ne peut plus rien faire."`

Contrôlez ensuite que le `Fighter other` est toujours en vie. Si ce n'est pas le cas indiquez : `"nom_other est déjà mort, ressuscitez-le afin de pouvoir le soigner."` Contrôlez également qu'il ait moins de 10 points de vie. Si ce n'est pas le cas, indiquez le via le texte suivant : `"nom_other a déjà le maximum de points de vie."`

Si toutes ces conditions sont réunies, ajoutez la valeur de l'attaque de l'instance qui appelle la méthode aux points de vie de `other`, puis appelez la méthode de classe `checkHealth()`.

Question 12: (🕒 5 minutes) Classe-fille `Attaquant`

Commencez par déclarer une nouvelle classe-fille `Attaquant` héritant de la classe `Fighter`. Cette classe-fille prendra un nouvel attribut privé nommé `multiplicateur` de type `int`, qui sera passé comme argument du constructeur de la classe-fille.

Déclarez le constructeur de cette classe ainsi que les `getter` et `setter` permettant d'interagir avec ce nouvel attribut `multiplicateur`.

Question 13: (🕒 10 minutes) Méthode `attack` de la classe-fille `Attaquant`

Réécrivez la méthode `attack` de la classe-fille `Attaquant` afin d'effectuer plusieurs attaques sur `other` en fonction de l'attribut `multiplicateur`.

Assurez-vous de toujours indiquer le type de l'attaque lorsque vous faites appel à la méthode `attack()`.

Si tout est correct, en utilisant ce `main` :

```
1 public class Main {
2     public static void main(String[] args) {
3         Fighter P1 = new Fighter("P1", 10, 2, 2);
4         Attaquant P2 = new Attaquant("P2", 10, 2, 2, 2);
5         Soigneur P3 = new Soigneur("P3", 10, 4, 2, 4);
6         P1.attack("pied", P2);
7         P1.attack("poing", P2);
8         P1.attack("tete", P2);
9         P1.attack("tete", P2);
10        P3.resurrection(P2);
11        P1.attack("pied", P2);
12        P1.attack("poing", P2);
13        P1.attack("tete", P2);
14        P3.attack(P2);
15        P2.attack("tete", P1);
16    }
17 }
```

Vous devriez obtenir :

```
1 P1 a encore 10 points de vie
2 P2 a encore 8 points de vie
3 P3 a encore 10 points de vie
4 -----
5 P1 a encore 10 points de vie
6 P2 a encore 6 points de vie
7 P3 a encore 10 points de vie
8 -----
9 P1 a encore 10 points de vie
10 P2 a encore 2 points de vie
11 P3 a encore 10 points de vie
12 -----
13 P2 est mort
14 P1 a encore 10 points de vie
15 P3 a encore 10 points de vie
16 -----
17 P2 vient de revenir à la vie
18 P1 a encore 10 points de vie
19 P3 a encore 10 points de vie
20 P2 a encore 10 points de vie
21 -----
22 P1 a encore 10 points de vie
23 P3 a encore 10 points de vie
24 P2 a encore 8 points de vie
25 -----
26 P1 a encore 10 points de vie
27 P3 a encore 10 points de vie
28 P2 a encore 6 points de vie
29 -----
30 P1 a encore 10 points de vie
31 P3 a encore 10 points de vie
32 P2 a encore 2 points de vie
33 -----
34 P1 a encore 10 points de vie
```

```

35 P3 a encore 10 points de vie
36 P2 a encore 6 points de vie
37 -----
38 Attaque n 1
39 P1 a encore 6 points de vie
40 P3 a encore 10 points de vie
41 P2 a encore 6 points de vie
42 -----
43 Attaque n 2
44 P1 a encore 2 points de vie
45 P3 a encore 10 points de vie
46 P2 a encore 6 points de vie
47 -----

```

3 Notions d'héritage - Python (15 minutes)

Question 14: (🕒 15 minutes) Classe Point (Suite)

Dans la semaine précédente, vous avez défini une classe `Point` représentant un point de 2 dimensions, avec des coordonnées x et y , ainsi que des opérations basiques sur des points 2D. Pour cet exercice, vous allez implémenter une classe de points à 3 dimensions qui héritera de la classe `Point` créée précédemment. Avant de commencer, modifiez les attributs privés de la classe `Point`. Remplacez les par des attributs protégés. Écrivez une classe qui hérite de `Point`. Nommez la `Point3D`. Après avoir rajouté la 3ème dimension comme attribut, effectuez les opérations ci-dessous :

- Rajoutez une méthode qui renvoie une représentation vectorielle du point. Vous pouvez utiliser une liste.
- Recalculez la distance euclidienne et le milieu pour le point 3D.
- **Pour aller plus loin** Si vous voulez vous familiariser encore plus avec les méthodes de classe en Python, implémentez deux autres algorithmes de calculs de distance : les distances de [Manhattan](#) et [Minkowski](#).

Vous pouvez compléter le code suivant (qui se trouve sur Moodle, dans le dossier Code).

```

1 class Point3D(Point):
2     def __init__(self, x, y):
3         super().__init__(x, y)
4         # Rajoutez un nouvel attribut z propre à la classe Point3D
5
6
7     # Définir le getter et le setter sur le nouvel attribut z
8
9     # Complétez les méthodes suivantes
10    def vector_representation(self): # représentée sous forme de liste
11        ...
12
13    def distance_euclidean(self, p2): # i.e norme
14        ...
15
16    def distance_manhattan(self, p2):
17        ...
18
19    def distance_minkowski(self, p2, order=3):
20        ...
21
22    def milieu(self, p2):
23        ...

```